

SkillSprint — Technical Architecture & Specification

Production-Grade System Specification for Modern EdTech Platform

Tech Stack: MERN (MongoDB, Express, React, Node)

Frontend Host: Vercel

Backend Host: Render

Document Version: v2.4 (2026 Release)

1. Executive Overview

SkillSprint is a next-generation Learning Management System (LMS) and Mentorship Portal engineered for high performance, smooth UX, and seamless administrative control. Designed using a modern full-stack architecture, it provides an intuitive course delivery system using **YouTube Unlisted API integration**, dynamic CMS control, and secure JWT-based access management.

2. UI/UX, Theme System & Design Language

The frontend is designed with a premium, sleek aesthetic utilizing Tailwind CSS, micro-interactions, and high-contrast accessibility standards.

DESIGN ELEMENT	SPECIFICATION	USER EXPERIENCE IMPACT
Primary Color Palette	#FF6B00 WARM ORANGE & #1A202C DARK SLATE	High brand recognition, energetic accent highlights, professional look.
Theme Engine	Light / Night Dual Mode (Persistent state via Next-Themes / Context API)	Seamless toggling in the header header navbar. Saved in `localStorage`.
Motion & Micro-Animations	Framer Motion (Hover scales, page route transitions, dynamic layout animations)	Provides fluid, desktop-app feel for web users. Smooth lock-to-unlock state changes.
Responsive Grid	Tailwind Mobile-First Grid (320px Mobile up to 1536px Ultra-wide screens)	100% adaptable layouts for smartphones, tablets, laptops, and wide monitors.

3. Architecture & Deployment Pipeline

Deployment Architecture Workflow

Frontend (Vercel): Next.js / React SSR + Axios + Tailwind CSS

|

└─▶ REST API Requests (CORS Enabled with Auth Headers)

Backend (Render - Express.js API): Node.js + JWT Auth Middleware + Cloudinary SDK

► Database: MongoDB Atlas (Cloud Managed NoSQL)

► Media Services: Cloudinary (Images) & YouTube API (Unlisted Video Streams)

4. Roles & Core Features Specification

A. Learner / User Role (Student)

- **Auth & Interactive Onboarding:** Responsive Modal Sign-Up / Login with form validation, visual password strength indicators, and JWT session handling.
- **Profile Management & Avatar Editor:** Dedicated profile setting page allowing users to update their full name, bio, target skills, password, and upload custom avatars (handled via Cloudinary).
- **Course Catalog & Dynamic Filters:** Real-time search, category dropdowns, and price sorting with instant client-side UI updates.
- **Enrollment & Payment Receipt Upload:** Modal workflow to select payment method, enter transaction IDs, and upload payment receipt images.
- **Gated Unlisted Video Player:**
 - **Locked State:** Displays a sleek dark backdrop with a centered Lock Icon  and prompt to complete enrollment.
 - **Unlocked State:** Unlocks embedded YouTube Player (`iframe`) as soon as Admin approves the enrollment status.

B. Admin Role (Platform Manager)

- **Course Management (CRUD):** Full controls to create, edit, update, or remove courses, lessons, durations, and YouTube unlisted video URLs.
- **Dynamic Site CMS:** Admin interface to change Hero Section titles, subtitles, banner background imagery, and featured announcement bars live.
- **Enrollment Verification Engine:** Admin Dashboard table listing pending payments with receipt previews, one-click **Approve** or **Reject** buttons, and instant user status synchronization.
- **Analytics Dashboard:** Visual stat cards detailing Total Revenue, Active Students, Enrolled Courses, and Pending Requests.

5. MongoDB Database Schema Specification

1. User Model Schema (`User.js`)

```
const userSchema = new mongoose.Schema({
  name: { type: String, required: true },
  email: { type: String, required: true, unique: true },
  password: { type: String, required: true },
  role: { type: String, enum: ['admin', 'learner'], default: 'learner' },
  avatar: { type: String, default: 'https://images.cloudinary.com/default-avatar.png' },
  bio: { type: String, default: '' },
  phoneNumber: { type: String, default: '' },
}, { timestamps: true });
```

2. Course & Unlisted Video Schema (`Course.js`)

```
const lessonSchema = new mongoose.Schema({
  title: { type: String, required: true },
  videoUrl: { type: String, required: true }, // YouTube Unlisted Embed URL
  duration: { type: String, default: '10:00 mins' },
  isPreviewFree: { type: Boolean, default: false }
});

const courseSchema = new mongoose.Schema({
  title: { type: String, required: true },
  description: { type: String, required: true },
  category: { type: String, required: true },
  price: { type: Number, required: true },
  coverImage: { url: { type: String, required: true }, public_id: { type: String } },
  lessons: [lessonSchema]
}, { timestamps: true });
```

3. Enrollment Verification Schema (`Enrollment.js`)

```
const enrollmentSchema = new mongoose.Schema({
  studentId: { type: mongoose.Schema.Types.ObjectId, ref: 'User', required: true },
  courseId: { type: mongoose.Schema.Types.ObjectId, ref: 'Course', required: true },
  status: { type: String, enum: ['pending', 'approved', 'rejected'], default: 'pending' },
  paymentProofUrl: { type: String, required: true },
  transactionId: { type: String, required: true }
}, { timestamps: true });
```

6. Backend REST API Endpoints Summary

METHOD	ENDPOINT	ACCESS	FUNCTIONALITY
POST	`/api/auth/register`	Public	Create student profile
POST	`/api/auth/login`	Public	JWT Token authentication
PUT	`/api/users/profile`	Authenticated	Update profile & avatar image
GET	`/api/courses`	Public	Fetch catalog with search/filter
POST	`/api/courses`	Admin Only	Create course + YouTube links
POST	`/api/enrollments`	Learner	Submit payment receipt
PUT	`/api/enrollments/:id/status`	Admin Only	Approve / Reject enrollment status